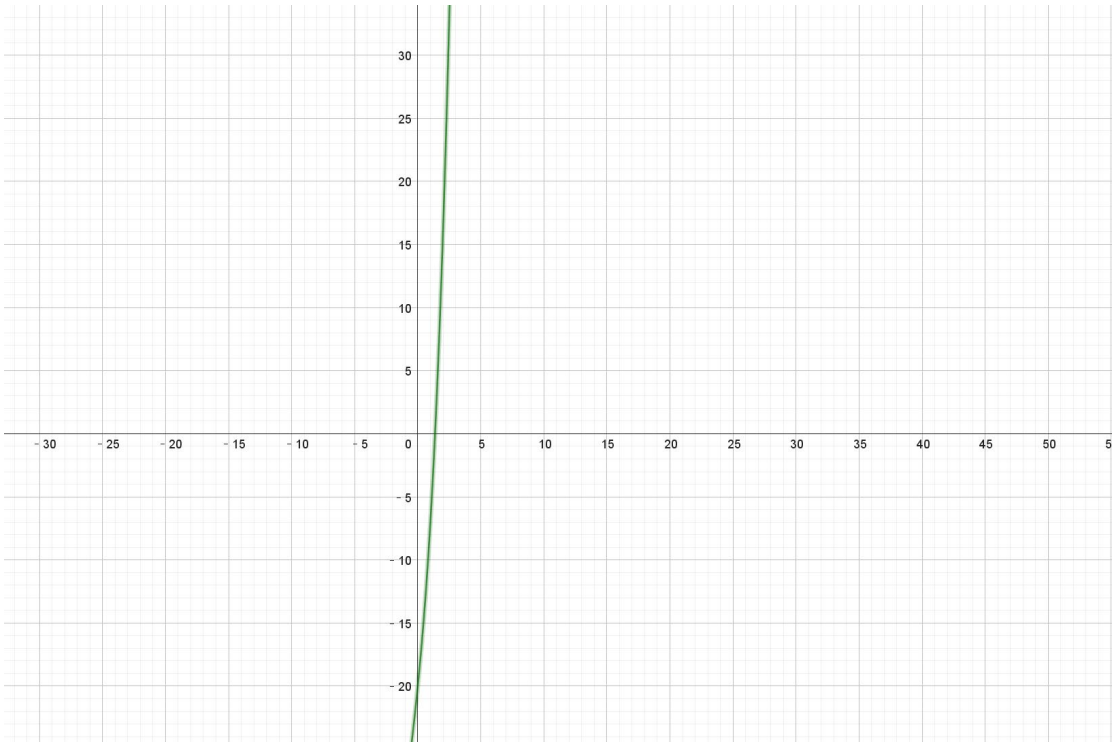
数值分析——非线性方程组上机报告

未央-水木 22 郁然

1 实验目的

本次上机作业需要用迭代法解决非线性方程组的求解问题。在求解过程中使用的方法有：不动点法、Steffensen 加速法和 Newton 迭代法。求解的方程为：$x^3+2x^2+10x-20=0$。

直观起见，使用 Geogebra 画出 $f(x)=x^3+2x^2+10x-20$ 的图像，观察图像的零点。由图像所知函数存在一个零点，计算器计算得到 $x^*\approx 1.368808107\cdots$。



计算可知，$f'(x)=3x^2+4x+10=3(x+\frac{2}{3})^2+\frac{26}{3}>0$，所以 $f(x)$ 为单调递增函数，故 x^* 为其唯一解。

我们期望通过这次作业，求出在 $x_0=1$ 处附近的解，比较几种方法直接效果与误差，并期望精度达到 10^{-7}。

2 实验方法详述

2.1 不动点迭代法

第一种不动点迭代方法，令 $\varphi(x)=\dfrac{20-2x^2-x^3}{10}$，$x_0=1$，我们希望迭代求出 $x=\varphi(x)$ 的不动点。

考虑 $\varphi'(x)=\dfrac{-3x^2-4x}{10}$，易知这是一个连续函数，$\left|\varphi'(x^*)\right|\approx 1.1096>1$，所以 $\varphi(x)$ 不具有局部收敛性，不动点迭代未必收敛。

第二种不动点迭代法，将原方程变形为 $x^3=2x^2+10x-20$，两边同开三次方得到迭代函数 $\varphi(x)=\sqrt[3]{-2x^2-10x+20}$。取 $x_0=1$，我们同样希望迭代求出 $x=\varphi(x)$ 的不动点。

同样考虑 $\varphi'(x)=\dfrac{-4x-10}{3(-2x^2-10x+20)^{\frac{2}{3}}}$，在 x^* 的邻域内连续，将 x^* 代入得到：$\left|\varphi'(x^*)\right|\approx 2.7532>1$。因此该迭代也有可能不收敛。

使用 matlab 软件处理并记录迭代的数据，为避免无限次迭代，取最大迭代数 $n=10000$，当达到上限时终止迭代。代码见附录 1 和附录 2。

2.2 Steffensen 加速法

用 Steffensen 加速法对上两个不动点迭代函数进行改进计算。具体算法如下：

我们令 y,z 分别满足 $y_k=\varphi(x_k),z_k=\varphi(y_k)$

那么随后一次迭代的结果 $x_{k+1}=x_k-\dfrac{(y_k-x_k)^2}{z_k+x_k-2y_k}$

只要满足 $\left|\varphi'(\alpha)\right|\neq 1$，那么 Steffensen 加速法至少是二阶收敛的。事实上，根据第一部分中的求导结果，两个函数均满足条件。

使用 matlab 软件处理并记录迭代数据，虽然这个算法收敛，我们仍取最大迭代数 $n=10000$，达到上限时终止迭代。代码见附录 3 和附录 4。

2.3 Newton 迭代法

我们记 $f(x)=x^3+2x^2+10x-20$ 根据 Newton 迭代法，我们有：

$$x_{k+1}=x_k-\frac{f(x_k)}{f'(x_k)}$$

首先验证 Newton 迭代法使用的条件。 $f''(x_k)=6x+4, f'(x)=3x^2+4x+10$，在 $[0,2]$ 上， $f(x)$ 有零点， $f''(x)$ 不变号，且有 $|f'(0)|=\min\{|f'(0)|,|f'(2)|\}$ 满足

$$10=\frac{|f(0)|}{2-0}=|f'(0)|$$

由此可知，牛顿迭代法二阶收敛到 x^*。

同样使用 matlab，具体代码见附录 5。

3 实验结果讨论

3.1 不动点迭代法

第一种令 $\varphi(x)=\frac{20-2x^2-x^3}{10}$ 时，迭代得到的结果为 $x=0.548946478\cdots$，这个结果与已知的解值相差甚远，将每次迭代的结果以散点图的形式展现，如图 1 所示。为了便于观察前期变化，我们选取前 300 项迭代结果，事实证明后续项的散点分布没有明显变化。

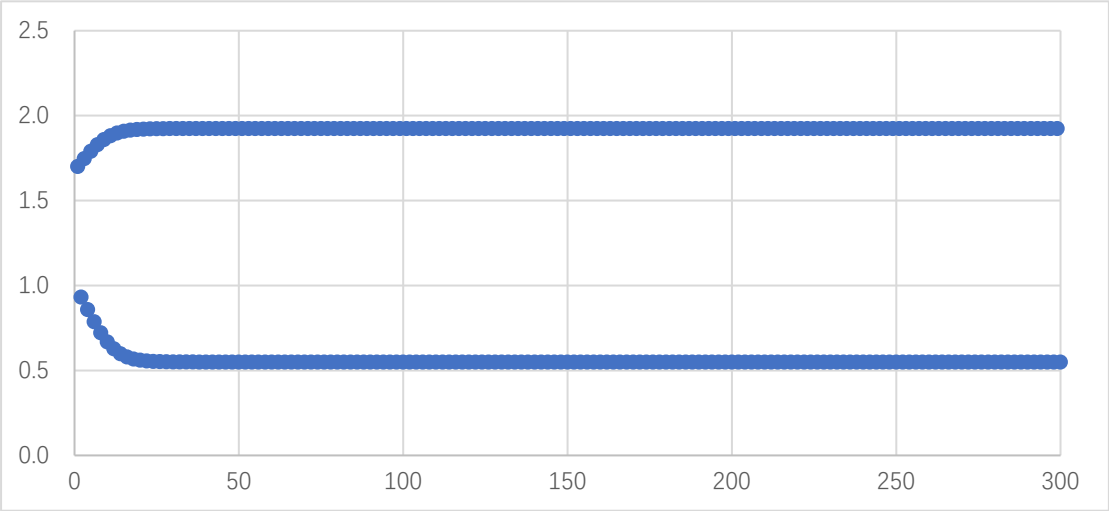


图 1 第一种不动点迭代法结果散点图

可以观察发现，最后迭代形成了 $f(x_1)=x_2, f(x_2)=x_1$ 的稳定平衡，故无法收敛到真正的根。

第二种不动点迭代的迭代结果为 $x=-3.162277660\cdots$，同样也与已知解值相差很大，用散点图显示每次迭代的结果如图 2 所示。同样我们也选取前 300 项结果，可以发现，最后迭代形成了 $f(x_3)=x_4, f(x_4)=x_3$ 的稳定平衡，无法收敛得到真正的根。并且第二个函数相比第一个更快达到平衡的状态。

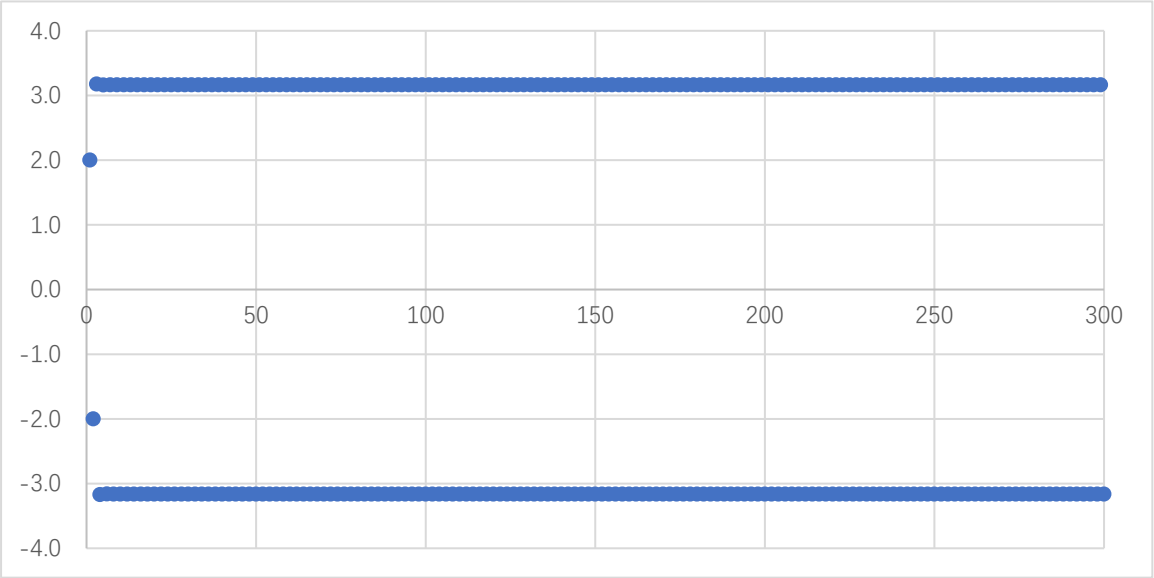


图 2 第二种不动点迭代法结果散点图

由结果我们可以得出结论，当 $|\varphi'(\alpha)|>1$ 时，使用 $x=\varphi(x)$ 很有可能无法得到精确的解。

3.2 Steffensen 加速法

我们对第一种不动点迭代法的函数采用 Steffensen 加速法进行改进，得到的每步数值如下表所示。

1.00000000000000
1.33349213911386
1.36841543910839
1.36880805830916
1.36880810782137
1.36880810782137

表 1 第一种函数 Steffensen 加速法

我们发现经过三次迭代就有了 7 位有效数字，而四次迭代有了 14 位有效数字。

对第二种不动点迭代法的函数采用 Steffensen 加速法进行改进，得到的每步数值如下表所示。

1.00000000000000
1.20000000000000
1.27374039954537
1.30830003900789
1.34727521973943
1.36672391381434
1.36878928488277
1.36880810628964
1.36880810782137
1.36880810782137

表 2 第二种函数 Steffensen 加速法

我们发现经过七次迭代得到了至少 7 位有效数字，八次迭代就有了 14 位有效数字。

两个函数相比第一个迭代的速度更快一点，这或许是因为$|\varphi'(\alpha)|$更接近 1 所造成的。因此我们合理猜测，导数绝对值越大，迭代时相对而言速度越慢。

我们对迭代的初值进行更改，记录得到至少七位有效数字时所需要的迭代次数，结果如下表所示。

初值	10	100	1000	-10	-100	-1000
次数	29	大于10000次	大于10000次	21	大于10000次	大于10000次

表 3 更改初值的第一种函数 Steffensen 加速法

初值	10	100	1000	-10	-100	-1000
次数	10	13	11	11	11	10

表 4 更改初值的第二种函数 Steffensen 加速法

可以发现，不同方法对于初值的要求不同。第一种方法当初值改变时迭代次数急剧增加，而第二种算法初值改变时迭代次数改变不明显。相似的是，当初值与精确解相近的时候，可以有效减少迭代次数。

3.3 Newton 迭代法

采用牛顿法得到的每步迭代数值如下表所示。

1.00000000000000
1.41176470588235
1.36933647058824
1.36880818861753
1.36880810782137
1.36880810782137

经过三次迭代就达有了 8 位有效数字，四次迭代有了 14 位有效数字。

由于牛顿迭代法需要满足一定的条件，所以对初值的范围有所限制，在初值范围内可以利用迭代较好地得到近似解。

4 实验总结

通过这次实验我们可以看出，利用不动点迭代法求解时，必须满足收敛的条件才可以保证迭代可以得到近似解。而 Steffensen 加速法虽然可以保证其迭代收敛，但初值的选取也会影响迭代的收敛速度，如果不加考虑初值范围也会造成运算上的麻烦。Newton 法在使用的过程中需要保证其约束条件成立，也需要对初值范围有大致的约束。

不同方法之间的比较我们可以看出，在使用迭代法求解非线性方程组的过程中，二阶收敛及更高阶收敛的方法具有较快的速度，可以节省计算的时间。同时，运算前我们也需要选取合适的初值，一个好的初值可以加速运算，提供更好的精度。

附录 程序代码

注：本次上机实验中，利用 matlab 完成相关操作，以下代码均为 matlab 中的代码。

1 第一种不动点迭代法

```
x=zeros(10000,1)
x(1,1)=(20-2-1)/10
for i=2:10000
x(i,1)=(20-2*x(i-1,1)^2-x(i-1,1)^3)/10
end
```

2 第二种不动点迭代法

```
x=zeros(10000,1)
x(1,1)=(20-10-2)^(1/3)
for i=2:10000
x(i,1)=nthroot((20-10*x(i-1,1)-2*x(i-1,1)^2),3)
end
```

3 第一种不动点迭代法的 Steffensen 加速法优化

```
function [a] = fai(b)
a = (20-2*b^2-b^3)/10
end
for j=2:10000
y(j,1)=y(j-1,1)-(fai(y(j-1,1))-y(j-1,1))^2/(fai(fai(y(j-1,1)))-
2*fai(y(j-1,1))+y(j-1,1))
end
```

4 第二种不动点迭代法的 Steffensen 加速法优化

```
function [a] = fai(b)
a = nthroot((20-10*b-2*b^2),3)
end
for j=2:10000
y(j,1)=y(j-1,1)-(fai(y(j-1,1))-y(j-1,1))^2/(fai(fai(y(j-1,1)))-
2*fai(y(j-1,1))+y(j-1,1))
end
```

5 Newton 迭代法

```
function [a] = newton(b)
a = b - (b^3+2*b^2+10*b-20)/(3*b^2+4*b+10)
end
for j = 2:100
x(j,1)=newton(x(j-1,1))
```

```
end
```